STUDY NOTES

KV Cache - 1

GQA, PagedAttention and Multi-head Latent Attention (MLA)

Detailed notes for a beginner level, built from the class recording and the tutor's board notes.

Class date	Sunday, 4 October 2026 (from the recording file name; confirmed by you)
Tutor	Paul
Recording	About 2 h 56 min, timestamps shown as [hh:mm:ss]
Level / style / language	Beginner / Detailed / English

Materials covered

File	What it is
Attention_Variants.pdf	23 pages of handwritten board notes (cited as "PDF p. N", numbered by page order)
GMT20261004-143211_Recording_cutfile_..._transcript.vtt	Speech-to-text transcript of the class (1,360 cues, 4 named speakers plus chat questions read aloud by the tutor)

Not covered in this session: RoPE (only a short preview) and the worked MLA matrix example. Both move to the next class.

Contents

How to read these notes 4

Topic Map 4

1. Class overview and roadmap 5

Agenda as written on the board (PDF p. 1) 5

2. Recap: MHA, MQA and the KV-head idea 5

Key concepts in simple words 5

3. Grouped Query Attention (GQA) 6

3.1 The idea 6

3.2 What changes and what does not 7

3.3 How much KV cache is saved 7

3.4 Fewer vs more KV heads: the trade-off 7

3.5 Choosing the number of KV heads: the design rule 8

3.6 Sharing ratio 8

3.7 Deciding KV heads from your GPU memory 8

4. PagedAttention (vLLM): smart KV cache management 10

4.1 What it is and where it comes from 10

4.2 The problem 10

4.3 The core idea: pages and a block table 11

4.4 Internal vs external fragmentation 12

4.5 How paging helps with fragmentation 12

4.6 The problems at a glance 12

4.7 Duplication and block sharing 12

4.8 Block hashing and prefix caching (PDF p. 16) 13

4.9 Reference counting (PDF p. 16-17) 13

5. Multi-head Latent Attention (MLA) 16

5.1 The problem MLA still tackles 16

5.2 The idea 16

5.3 The numbers (per token) 16

5.4 How MLA works: compress, cache, expand 16

5.5 What changes and what does not 17

5.6 Quality and history 17

5.7 RoPE preview (not taught yet) 18

5.8 KV-cache quantization 18

6. Self-hosting and justifying GPU memory (MLOps question) 20

7. Key takeaways **21**

8. Glossary **22**

9. Action items, homework and logistics **22**

10. Resources and tools mentioned **23**

11. Unresolved or unclear points **23**

12. Conflicts and ambiguities log **23**

13. Quick revision cheat sheet **25**

How to read these notes

- **Order:** the notes follow the logical study order (prerequisites first), not the order of the transcript. Every mention of a topic across the whole class is merged into one place.
- **Timestamps** such as [00:23:10] point to the transcript. **PDF p. N** points to the board notes.
- **Conflicts:** where the tutor, the board or the transcript disagree, the notes **use the best-supported version** and show the conflict in an orange box. All of them are also collected in the Conflicts and Ambiguities Log near the end.
- **[unclear: ...]** marks speech-to-text or handwriting that could not be confirmed. Obvious speech-to-text mistakes (for example "page retention" for PagedAttention) are silently corrected.

> **KEY CONCEPT**
> Callout colours: **Key Concept** (blue), **Example** (green), **Tip** (amber), **Warning** (red), **Q&A** (purple), **Extra context** (grey, not from class), **Conflict / Ambiguity** (orange).

Topic Map

The tutor mostly taught in a sensible order, so the study order below matches the taught order. The only difference is from the printed agenda (PDF p. 1), which listed MLA second, but PagedAttention was taught before MLA.

#	Topic (study order)	Where in the class	PDF pages
1	Recap: MHA, MQA, KV-head idea	[00:05:44]-[00:09:27]	p. 1
2	GQA: idea, memory maths, design rule, sharing ratio	[00:09:27]-[00:36:20]	p. 2-5
3	GPU memory budget for the KV cache (revisited at the end)	[00:36:46]-[00:42:06], [02:53:22]-[02:54:40]	p. 5
4	PagedAttention (vLLM): fragmentation, blocks, sharing, hashing, reference counting	[00:42:48]-[02:08:27] (break at [01:31:24]-[01:44:00])	p. 6-17, 23
5	MLA (DeepSeek): latent compression and expansion	[02:08:32]-[02:40:00]; Q&A [02:44:49]-[02:51:25]	p. 18-22
6	RoPE: short preview only	[02:40:00]-[02:41:11]	p. 22 (crossed-out note)
7	Logistics, roadmap, general Q&A	scattered: [00:01:42]-[00:04:17], [01:32:23]-[01:43:28], [02:41:11]-[02:56:29]	none

Revisited topics (merged): the GPU memory split (topics 2 and 7), "PagedAttention works with every variant" (topics 4 and 5), and KV-cache quantization (topic 5).

1. Class overview and roadmap

Summary. This session continues the series on the "attention variants" that shrink or manage the KV cache (the stored keys and values that let an LLM avoid recomputing the past). It finishes GQA, teaches PagedAttention, and starts MLA.

Agenda as written on the board (PDF p. 1)

Planned	Status at the end of class
1) Grouped Query Attention (GQA)	Done
2) MLA (DeepSeek)	Idea, numbers and compress/expand pipeline done; worked matrix example and the RoPE link postponed
3) Paged Attention (KV cache management)	Done (taught before MLA)
4) RoPE (leads into MLA)	Not started; one-paragraph preview only

- Paul says this is a **theory-heavy** session with little practical work, because these are the internal components of every modern architecture [00:04:23].
- Timeline given: the GQA paper came in 2023, then MLA arrived in DeepSeek models [00:04:37].
- Paul warns that PagedAttention is **not** "another attention mechanism like the others"; it is about KV cache management, so its purpose is different [00:04:01].
- He considered teaching RoPE before MLA because the DeepSeek paper brings RoPE into MLA, but chose PagedAttention first because it is an easier and widely used idea [00:42:22].
- If you missed the previous session, review it first: this class assumes MHA, MQA, the KV cache, prefill/decode and the "KV memory" formula [00:01:15].

> **TIP**
> Paul's study advice: keep your own notes (handwritten, AI-generated or self-made) so revision is fast later; "spend the time once in your life" to learn these concepts [01:36:00], [01:38:25].

2. Recap: MHA, MQA and the KV-head idea

Summary. Every variant in this class is an answer to one question: how few K and V heads can we store without hurting quality? MHA stores one K,V per head; MQA stores one K,V in total.

Key concepts in simple words

> **KEY CONCEPT**
> **Head.** One "view" of the sentence. Each head has a query (Q), a key (K) and a value (V).
>
> **MHA (Multi-Head Attention).** With 8 heads, H1 uses Q1,K1,V1 ... H8 uses Q8,K8,V8. Every head has its own K and V [00:06:19]. (Paul first wrote "3" instead of 8 on the board and corrected it [00:07:15].)
>
> **MQA (Multi-Query Attention).** All 8 heads share **one** K1,V1 (a single KV head). Q1 ... Q8 stay separate [00:07:40].
>
> **Goal of the whole series:** reduce the number of **K and V heads**, because those are what is stored in the KV cache [00:06:54]. Queries are never affected [00:09:22].
>
> **Attention diversity.** Different heads can notice different patterns and relationships. With only one KV head you capture fewer patterns, so quality can drop [00:08:33].

Text diagram from PDF p. 1

```
MHA (8 heads)                    MQA (8 heads)
H1 -> Q1, K1, V1                 H1 -> Q1, K1, V1
H2 -> Q2, K2, V2                 H2 -> Q2, K1, V1
 :                                :
H8 -> Q8, K8, V8                 H8 -> Q8, K1, V1
8 KV heads (max memory)          1 KV head (min memory, less diversity)
```

ADDITIONAL INFO FROM MATERIALS

PDF p. 1 writes "Reduce: KV Heads" and "one KV head shared", and notes the cost as "Attention Diversity: losing".

3. Grouped Query Attention (GQA)

Summary. GQA is the middle ground between MHA and MQA: several query heads share one K,V pair, so you store more KV heads than MQA (better quality) but fewer than MHA (less memory). The number of KV heads is a hyperparameter chosen by the model designer.

3.1 The idea

- MQA goes straight from 8 KV heads down to 1, so a lot of information is lost. GQA wants to **lose less** [00:09:58]-[00:10:33]. PDF p. 2: "Middle ground (MHA - MQA); reduce the loss of info".
- Method: split the query heads into groups. All queries in a group share one K,V pair. How many KV heads to keep is "totally up to you" [00:10:44].
- Queries stay exactly as they are; only the K and V side changes [00:12:30].

Grouping example (assumption made by the tutor to build intuition)

```
8 query heads, groups of 2  ->  4 KV heads  (PDF p. 2)
  H1 Q1 \
  H2 Q2 /--> K1 V1
  H3 Q3 \
  H4 Q4 /--> K2 V2
  H5 Q5 \
  H6 Q6 /--> K3 V3
  H7 Q7 \
  H8 Q8 /--> K4 V4
KV heads: MHA = 8, GQA = 4, MQA = 1
```

KEY CONCEPT

In one line: GQA = Q heads unchanged, KV heads reduced by sharing within groups. It increases attention diversity compared with MQA, but less than MHA [00:12:19].

Paul stresses that the grouping choice (2 per group, 3 per group, etc.) is only an assumption to build the analogy and "can be different" [00:11:22].

Unequal groups of 3 (PDF p. 3)

Paul also drew groups of three (H1-H3, H4-H6, H7-H8) to show that the grouping is a choice [00:18:24]. But with 8 heads, groups of 3 do not divide equally, so he would choose 2 or 4 instead. Equal distribution is the preference [00:19:48].

3.2 What changes and what does not

	MHA	MQA	GQA (8 heads, 4 KV heads)
Number of heads	8	8	8 (unchanged)
K,V dimension per head	64	64	64 (unchanged)
Are K,V values different per head?	Yes, all different	No, all the same	Same inside a group, different between groups
KV heads stored	8	1	4

- The **dimension must stay the same** as Q, otherwise the Q and K dot product cannot be computed. Only the **values** are reused again and again [00:12:53], [00:14:20].
- The **number of heads does not change** either. What changes is how many distinct K,V sets exist [00:14:53].

3.3 How much KV cache is saved

Compare the number of KV heads with MHA. With 8 heads and 4 KV heads, GQA stores half: a **50% reduction** in KV cache versus MHA [00:15:20]-[00:17:40]. Paul also calls this "50% info loss" as a rough intuition: what you save you also give up [00:16:12]. For MQA versus MHA (8 heads) the answer is about **12.5%** (participants guessed 90%) [00:18:10], PDF p. 2.

PDF p. 3 then moves to a bigger example with 32 heads, and says all counts below are **KV heads**, because we are comparing KV memory [00:20:26]-[00:23:12]:

Variant	KV heads	KV memory (share of MHA)
MHA	32	100%
GQA	16	50%
GQA	8	25%
GQA	4	12.5%
MQA	1	3.12% (1/32 = 3.125%)

CONFLICT / AMBIGUITY

"12.5%" is used in two senses. For GQA the tutor says "50%" meaning the **reduction**. For MQA (8 heads) he says "12.5%" meaning what **remains** (1 of 8). In the 32-head table every percentage is the share **remaining**. *Used in these notes:* the table values are "share of MHA memory that remains"; the reduction is 100% minus that (for example GQA with 16 KV heads: 50% remains, so 50% reduction). This subtraction is simple arithmetic from the class numbers.

Also, the class starts from 8 heads ([00:20:26]) and then switches to 32 ("let's use a bigger number" [00:21:01]). The 32-head table on PDF p. 3 is the one used here.

3.4 Fewer vs more KV heads: the trade-off

- **Fewer KV heads** mean more memory efficiency (smaller KV cache), but less attention diversity and less expressive representations, so possibly lower quality [00:23:12]-[00:23:54], [00:30:46].
- **More KV heads** mean more independent representations in a high-dimensional space, so patterns and relationships are easier to capture, but a bigger KV cache.
- GQA was designed so that you "do not solve the entire problem" but avoid the extreme problem of MQA [00:23:54]. Nobody really wants exactly one KV head.

- The real question for a designer: "How much quality am I willing to trade for inference efficiency?" The benefit lands directly on the KV cache [00:30:08]-[00:30:38].

3.5 Choosing the number of KV heads: the design rule

KEY CONCEPT

Practical design rule: $H_{KV} = H_Q / 4$. That is, **for every 4 query heads, use 1 KV head** (PDF p. 4, [00:24:56]-[00:25:44]).

Large models (the tutor names 7B and 13B models) commonly have about **32** query heads, so 32 / 4 = **8 KV heads** [00:24:33]. (He also says "28" after 13B; that is [unclear: possibly 28B or another size].)

EXAMPLE

Design example (PDF p. 4, [00:27:30]-[00:29:05]): hidden dimension = 4096; query heads = 32; head dimension = 128. KV heads = 32 / 4 = **8**. Always start from the number of query heads: "that is the most important part of the design".

Condition	Result
$H_{KV} = H_Q$ (e.g. 32 Q and 32 K)	GQA behaves exactly like **MHA**
$H_{KV} = 1$ (32 Q and 1 KV)	GQA behaves exactly like **MQA**

- **Not a universal optimum.** The ratio 4 is a middle-ground default that most designers follow because the quality impact is good and KV memory shrinks a lot [00:25:44]-[00:26:04]. Start with it and adjust up or down [00:26:42].
- Other hyperparameters of a transformer architecture: number of layers, hidden dimension, number of attention heads, head dimension [00:27:01].
- **Whole numbers only.** Avoid unequal groupings. If H_Q is not divisible by 4 (the tutor's hypothetical is 27), change the denominator (for example to 3, giving 9) so the result is a whole number, never something like 4.57. Odd numbers of KV heads are fine [00:31:55]-[00:33:14], [00:35:46].
- Powers of two (such as 32) are favoured because hardware is optimized for them (the "2 to the power n" rule) [00:32:08].

3.6 Sharing ratio

KEY CONCEPT

Sharing ratio = Q heads / KV heads (PDF p. 5). It tells you how many query heads share one KV head. A ratio of **4** is the "golden rule" [00:33:21]-[00:33:54], [00:35:26].

It is the same formula as the design rule, just named. It is chosen by the LLM architect and decides "how much information is shared" [00:34:06]-[00:34:54]. Paul says this term can come up in interviews [00:33:42].

You can break the golden rule if the layer definition changes (for example, very different Q-head counts) [00:35:46].

Training note: models are **trained** with MQA, GQA (and, later, MLA) built in; the DeepSeek architecture was trained with MLA [00:34:57]-[00:35:26].

3.7 Deciding KV heads from your GPU memory

A participant asked whether going from 50% to 25% KV memory depends on memory availability or on performance. Paul's answer: look at the **KV-cache part** of GPU memory, not total memory [00:36:46]-[00:38:43].

EXAMPLE

A100 with 40 GB VRAM (PDF p. 5, [00:37:03]-[00:38:43]): about 65% goes to model weights, leaving about 35%. Roughly 5% is kept for other processes (so the board shows 30% for the KV cache, about 5% others). Paul says 30% of 40 GB is about **12 GB for the KV cache** [00:41:46].

If the GPU changes, the numbers change: he mentions an 80 GB A100 or H100 and a 140 GB H200 [00:41:46].

Practical advice: use about 95-96% of the GPU and keep 4-5% free in case something else runs; you could leave only 1-2% if you want more cache [00:40:38]-[00:40:55]. The goal is to avoid **OOM** (out of memory), the most common deep-learning error along with shape mismatch [00:40:59]-[00:41:35].

GPU memory split (illustrative)

```
A100 40 GB VRAM (PDF p. 5)
|-------- model weights ~65% --------|--- KV cache ~30% ---|-5%-|
                                                    (~12 GB)  others
if the total exceeds the card  ->  OOM  (in training and in inference)
```

CONFLICT / AMBIGUITY

Two different splits were mentioned. The worked example (board, [00:37:35]) is 65% weights / 30% KV cache / 5% others. Later, while answering an MLOps question, Paul says "out of 100%, 30% will be used by the model, I am keeping 50% for the KV cache" [02:53:59]. *Used in these notes:* 65/30/5, because it is the detailed example drawn on the board; the later numbers read as a hypothetical to illustrate "justify your GPU".

Paul says the 65% was taken "for Llama, I think the Llama 3, that 13 billion model" and is not a standard rule [00:39:34]. He sounded unsure of the exact model, so treat it as an illustration.

Q&A

Q. Dimensions of K and V in GQA? Are they smaller? [00:12:45]-[00:14:41]

A. No. They stay equal to the Q dimension (64 in the example) so matrix multiplication still works. Only the values are reused across a group.

Q. By how much does GQA reduce KV cache versus MHA, and MQA versus MHA? [00:17:15]-[00:18:24]

A. GQA (8 to 4 heads) is a 50% reduction. MQA is about 12.5% of MHA with 8 heads (not 90%).

Q. Hypothetical: what if the number of heads is a prime number? [00:34:23]

A. Paul says research rarely does that, you can use odd numbers, but the final KV count must be a whole number. His remark about reusing the same prime number is [unclear].

Q. Do we train with GQA? [00:34:57]

A. Yes. GQA, MQA (and MLA) are trained into the architecture.

Q. Is it necessary to know the underlying details? [00:36:27]

A. Paul: if you do not want to know the internals, skip that part; "otherwise you can skip the entire transformer". (Answer partly garbled.)

Q. Does it depend on memory availability or performance? [00:36:46]

A. Look at the KV-cache memory available on your GPU (the 65/30/5 example), then pick KV heads to fit.

Q. Can we change a model from MHA to GQA during fine-tuning? [00:38:43]

A. No. Fine-tuning uses an existing model whose architecture is fixed; changing it would mess up the learned weights.

Q. Is 65% for weights a standard rule? [00:39:18]

A. No. It was taken for Llama in this example; it depends on the model.

Q. What does OOM stand for? [00:41:18]

A. Out of Memory. Together with shape mismatch it is the most common deep-learning error.

ADDITIONAL INFO FROM MATERIALS

Head dimension = **128** in the design example is only on PDF p. 4; Paul did not say it aloud ("let's take a bigger number" [00:28:03]).

PDF p. 3 shows the explicit 32-head memory table; PDF p. 5 shows the A100 bar diagram labelled "Inference Efficiency" and the OOM arrow pointing to both training and inference.

SAID IN CLASS, NOT IN MATERIALS

A hypothetical 27-head example, the "whole number" rule, the 2^n hardware reason, training with GQA, and that fine-tuning cannot swap MHA for GQA.

4. PagedAttention (vLLM): smart KV cache management

Summary. PagedAttention stops allocating one huge contiguous memory block per request. Instead the KV cache is split into small fixed-size blocks ("pages") that can sit anywhere in GPU memory, tracked by a block table. This cuts wasted memory and lets requests share identical blocks.

4.1 What it is and where it comes from

- Comes from the open-source **vLLM** inference project [00:43:11]. The idea is borrowed from **virtual memory / paging in operating systems** [00:57:21].
- Purpose: efficient KV cache management during **inference**, not a new way to compute attention [00:04:01], [00:44:20].
- Terms used interchangeably: **page = block = chunk** [00:58:22].
- It works with **any** variant: MHA, MQA, GQA or MLA (PDF p. 23, [02:37:09]-[02:37:41]).

4.2 The problem

Imagine several users served by the same model at once. Every request needs its own KV cache in GPU memory [00:44:29]-[00:45:48]. You cannot allocate memory blindly, and the naive way is to reserve one **contiguous** block sized for the **maximum** output or context window (the example uses **4096** tokens) [00:46:08]-[00:47:34].

EXAMPLE

PDF p. 6: Request 1 (user A) uses 2000 tokens, request 2 (B) 500, request 3 (C) 300, request 4 (D) 1000 [00:50:11]. Each has a 4096-token slot reserved, so most of it sits unused.

Usage of the 4096 slot, computed from those numbers: A about 49%, B about 12%, C about 7%, D about 24%.

CONFLICT / AMBIGUITY

For B (500 tokens) Paul says "20% usage" [00:48:04]; 500 / 4096 is about **12%**. *Used in these notes:* the computed 12%. His 50% for A (2000 tokens) is correct to rounding.

Problem 1: fragmentation

When a request finishes (say C, which only needed 300 tokens), its memory is released and leaves a "hole" [00:51:30]. A new request F that needs 2048 tokens cannot use a 300-token hole [00:53:18]-[00:54:29]. The freed memory exists but is too small, so it stays unused.

Problem 2: unknown final sequence length

You never know how long the output will be. A request may start with a 500-token prompt and generate 5,000 tokens [00:56:03]. Reserving a huge contiguous block upfront wastes memory; Paul compares it to "booking an entire hotel for one guest" [00:56:28]. Hence contiguous allocation must go [00:56:47]-[00:57:06].

CONFLICT / AMBIGUITY

How many problems? The board (PDF p. 7) lists two: (1) fragmentation and (2) unknown sequence length. Later Paul says there are three: internal fragmentation, external fragmentation and duplication [01:14:04], [01:21:07]. *Used in these notes:* fragmentation (internal + external), unknown sequence length, and duplication. Paul never states explicitly how paging answers the unknown-length problem; it is implied by allocating small blocks only as needed (listed under Unresolved points).

4.3 The core idea: pages and a block table

- Break the KV cache into **small, fixed-size pages/blocks**. Example with 16 tokens per block: Page 0 holds tokens 0-15, Page 1 tokens 16-31, Page 2 tokens 32-47, Page 3 tokens 48-63 (PDF p. 7, [00:58:48]-[00:59:44]).
- Pages do **not** have to be physically next to each other. In the logical view the pages may be numbered 7, 1, 13, 11 (PDF p. 8) and live anywhere [00:59:55]-[01:01:31].
- A **GPU block table** maps each logical block to a physical GPU block: Block 0 maps to GPU block 7, Block 1 to GPU block 2, Block 2 to 15, Block 3 to 4 (PDF p. 8, [01:01:57]-[01:02:47]).

Block table (PDF p. 8)

```
Logical (what the sequence sees)     GPU block table        Physical GPU
Block 0 : tokens  0-15      ------>  GPU block 7           blocks anywhere
Block 1 : tokens 16-31      ------>  GPU block 2           in memory,
Block 2 : tokens 32-47      ------>  GPU block 15          not in order
Block 3 : tokens 48-63      ------>  GPU block 4
```

WARNING

Do not read "Block 0 = GPU block 0". The numbers are different on purpose; the table just says where each logical block physically sits [01:03:45]-[01:04:07]. GPU block allocation is effectively random [01:06:01].

How attention reads the blocks (PDF p. 9)

Keys and values for all tokens are K = [K1 ... Kn] and V = [V1 ... Vn]. With blocks, GPU block 7 might hold K1-K16, block 2 holds K17-K32, block 15 holds K33-K48, block 4 holds K49-K64. The **GPU kernel uses block mapping to retrieve** them when attention runs [01:08:32]-[01:10:55]. K and V can be stored together in a block (good practice) or separately; both are fine [01:11:25]-[01:12:01].

- The serving framework (**vLLM, SGLang**) takes care of all this. You mainly choose the model and how much GPU memory goes to the KV cache versus other processes [01:13:09]-[01:13:50].
- Paul says this theory is a "one-time thing" to know, especially for interviews, because afterwards you simply use vLLM [01:27:43].
- The context window (for example 4096) is fixed when you serve the model and applies to every user [01:12:46]-[01:12:59].

CONFLICT / AMBIGUITY

Some token indices on the board and in speech are off by one (for example the last key written as K64 vs "K63"; Paul himself says "sometimes I write wrong" [01:09:19]). The idea is unchanged: each block holds 16 consecutive keys (tokens 0-15, 16-31, ...).

4.4 Internal vs external fragmentation

Type	Meaning	Example
Internal (IF)	Allocated memory is larger than what you need.	Request has 10 tokens but a 16-token block is allocated; slots 11-16 stay empty [01:15:13]-[01:16:23].
External (EF)	Total free memory is enough, but scattered in separate holes.	Requests A and C are using memory; B and D are free (PDF p. 10). Free units total 9, a new request needs 10, so it cannot start and usually waits in a queue [01:16:43]-[01:20:37].

4.5 How paging helps with fragmentation

External fragmentation. Blocks need not be contiguous, so scattered free blocks become useful: they can be combined as one logical sequence (PDF p. 11-12, [01:23:12]-[01:26:46]). Without paging you needed one big contiguous region; with paging each block can go to any free block [01:25:08]-[01:26:20]. It is **reduced, not fully solved**: if the total free memory is truly too small, nothing helps, and a block must actually be free to be used [01:22:03]-[01:24:42].

Internal fragmentation. Waste is limited to the **last, partly filled block**. Example (PDF p. 12): block size 16 tokens, a sequence of 18 tokens. Block 1 holds 16 tokens, Block 2 holds 2, so **14 token slots are wasted**. Full blocks waste nothing, so the waste is at most about **one block per sequence** [01:28:00]-[01:30:45]. It is reduced, not eliminated.

KEY CONCEPT

Summary of what paging achieves: small waste per sequence (internal) and usable scattered memory (external). Neither problem disappears, because both are real memory limits [01:22:13]-[01:22:35].

4.6 The problems at a glance

Problem	What PagedAttention does	Solved?
Internal fragmentation	Small fixed blocks, waste only in the last block	Reduced, not eliminated
External fragmentation	Non-contiguous blocks, scattered free blocks reusable	Reduced, not eliminated
Unknown sequence length	Memory is held in small blocks instead of one huge upfront block (implied)	Not stated explicitly
Duplication	Block sharing / prefix caching, tracked by hashes and reference counts	Addressed (next sections)

4.7 Duplication and block sharing

The problem. If users A and B send the **same prompt**, the prefill stage computes identical K,V for both, yet a naive system stores them **twice**: "same info, twice" [01:45:03]-[01:46:15] (PDF p. 13).

The fix: block sharing (page sharing). Because the KV cache is in blocks, several sequences can point to the **same physical block**. For example users A and B both point to KV block 17, instead of A using block 17 and B block 25. **One physical copy serves both requests** [01:46:37]-[01:48:20].

This is called **prefill sharing / prefix sharing**, in other words KV sharing [01:48:34]-[01:49:42]. Recall the prefill phase from the previous class: it computes and saves the KVs, but now into shared blocks [01:49:12]-[01:49:35].

Chat systems: the system prompt (PDF p. 14-15)

- Every chat system has a **system prompt (SP)** [01:50:11]. Each user request is SP + that user's prompt: A = SP + prompt A, B = SP + prompt B, C = SP + prompt C [01:50:42]-[01:51:05].
- The SP part is identical, so its KV blocks are **shared** by all users. After the shared blocks, each user **diverges** into new, user-specific blocks (A1, B1, C1) [01:52:19]-[01:53:13].
- Partial overlap works the same way. Example: a 100-token prompt where A, B and C share 50 tokens and differ in the other 50: the shared 50 tokens use the same physical blocks [01:58:27]-[01:59:02]. If the whole prompts are identical there is no divergence at all [01:55:15]-[01:56:01].

EXAMPLE

A 2K-token system prompt shared by two concurrent chats is stored **once** [02:03:15]. "Cache once, use 200 times" [01:56:38].

How long to keep the cached system prompt

- Keep the shared KV in memory; do not clear it casually, otherwise it must be recomputed [01:56:20]. You cannot hold it forever: clear when the cache is nearly full (for example about **95% GPU utilization**) or after roughly 500-1000 requests [01:56:29]-[01:57:19].
- There is **no TTL / expiry**; it is your decision based on number of requests and GPU utilization. Track usage with tools such as **nvtop** or other GPU monitors [01:57:39]-[01:59:16].
- Traffic example: a system is busy 16-18 hours and nearly idle for 6. In the busy period the cache fills quickly so you may clear and recompute every 1-2 hours; when traffic is low it fills slowly and can be held longer. With a huge VRAM (for example 300 GB) it matters less, but long generations (up to 1M tokens) force clearing [02:00:52]-[02:03:03].

WARNING

Sharing the KV cache does **not** make two users' **outputs** identical. Only the prefill (prompt) KV is reused. LLM generation (decode) is non-deterministic and each user can get a different answer [01:59:16]-[02:00:16].

4.8 Block hashing and prefix caching (PDF p. 16)

KEY CONCEPT

Each block gets a **unique hash**. Same block content means same hash means same prefix, so a new request can find a reusable block by matching hashes [02:03:45]-[02:05:12], [02:06:00].

Names you will see for this idea: **block-level hashing, prefix hashing, prefix caching, block-level caching, KV-cache sharing**. Use whichever term you like, but know they are the same family [02:04:35]-[02:05:44].

4.9 Reference counting (PDF p. 16-17)

If A and B share a block (say block 17), the block cannot be freed when A finishes, because B still uses it. PagedAttention keeps a **reference count** per block [02:06:27]-[02:07:30].

Reference counting (PDF p. 17)

```
users A and B share the same prompt block
reference count = 2   (A and B)
A finishes            ->  count = 1   (block must stay)
B finishes            ->  count = 0   (block can now be released)
```

CONFLICT / AMBIGUITY

Paul's spoken walk-through of the counter is garbled ("the count becomes true... then becomes 1... then zero") [02:07:58]-[02:08:13]. *Used in these notes:* the board version 2, then 1, then 0.

He says this is internal bookkeeping done by PagedAttention: "I have not told you about it, but remember" [02:06:27].

Q&A

Q. Why not keep clearing memory after each request? [00:55:36]

A. The memory is cleared; the problem is that the freed piece is not large enough for the next request.

Q. Can we control the number of tokens? [00:49:05]

A. Yes, it is a simple parameter, but it is limited by the context window (you cannot generate 5,000 tokens with a 4096 window).

Q. Does Q also take GPU memory, apart from the KV cache? [01:07:23]

A. Paul turned it back to the class: "why do you need to save Q at all?" and noted that Q is the query matrix, not the user query. He did not state a final answer (see Extra context).

Q. Why is block 7 in GPU block 0? [01:03:45]

A. It is just a random example; logical block 0 does not map to GPU block 0.

Q. If we break the memory evenly, will it work? [01:05:37]

A. Even breaking is not strictly necessary; hardware prefers powers of two (2^n), and breaking that pattern is where problems start.

Q. Is GPU block allocation random? [01:06:01]

A. Yes, GPU blocks do not depend on the PagedAttention block numbers.

Q. Will R1, R2, R3 be divided into multiple pages? [01:06:11]

A. Yes, multiple pages/blocks per request.

Q. Can we use LRU for managing pagination? [01:06:20]

A. Yes, but "a different alternate is there" (not named in class).

Q. Does unused memory go back to the pool? [01:06:31]

A. Yes, it is released; whether it is enough for the next request is the issue.

Q. Is the 4096 context window per user? [01:12:46]

A. No. The model's context window is fixed once at serving time and is the same for every user.

Q. Do we program the hardware, or define it in the model? [01:13:09]

A. Neither. Serving frameworks (vLLM, SGLang) handle it; you pick the model and memory split.

Q. How does KV cache relate to the number of requests and volume? (Pandian) [00:44:02]

A. Deferred: "it will become clearer after the topic". It is covered by the multi-user problem in 4.2.

Q. Shall we use CPU for external fragmentation? [01:34:33]

A. No.

Q. Can two requests share the same physical KV block? [01:54:03]

A. Yes, physical blocks can be shared.

Q. When a sequence completes, what happens to its blocks? [01:54:24]

A. They are cleared and join the scattered free memory, which new requests can use if the total is enough; if the request needs more than is free, it cannot be served.

Q. If A and B prompts share only some tokens? [01:55:03]

A. Shared blocks are reused, then the sequences diverge. If the whole prompt is identical, there is no divergence.

Q. Is a required level of token similarity needed? [01:56:01]

A. You do not need to manage that yourself.

Q. If the cache was erased after user A, and B asks the same? [01:57:19]

A. Then it must be recomputed.

Q. Does the cache expire (TTL)? [01:57:39]

A. No TTL. You clear it when utilization nears about 95%.

Q. If 3 prompts have the same initial tokens, are the blocks shared? (Vivek) [01:58:08]

A. Yes.

Q. How do we know the cache is at 95%? [01:59:07]

A. Use nvtop or other GPU tools to track processes.

Q. Will sharing blocks make the answers similar? [01:59:16]

A. No. Only the prompt KV is shared; generation is non-deterministic.

Q. Why clear the cached system prompt at all? (Subhash) [02:00:33]

A. Memory is finite and you cannot hold it for a lifetime; the traffic example above explains when to clear.

Q. Do providers such as Gemini and OpenAI clear caches like this? [02:03:24]

A. Yes presumably, but they do not publish their serving strategies.

Q. What do you mean by "profile"? [02:05:53]

A. Paul never used that term; he confirmed that matching same blocks by hash is correct and that prefill means computing and saving the KVs.

Q. Is PagedAttention possible on top of MLA? (Bhaskar) [02:37:09]

A. Yes; it is independent of MHA/MQA/GQA/MLA.

EXTRA CONTEXT (NOT FROM CLASS)

In standard decoder inference, only past **keys and values** are reused from the cache; the query for the newest token is computed fresh each step, which is why it is not cached. This is background knowledge to clarify the unanswered question about caching Q.

ADDITIONAL INFO FROM MATERIALS

PDF p. 9 handwriting mentions "vLLM / SGLang" and "Kernel: block mapping to retrieval". PDF p. 7 labels the intuition "KV cache = small fixed size pages/blocks".

SAID IN CLASS, NOT IN MATERIALS

Hotel analogy; the clearing policy (95%, no TTL, nvtop); non-determinism of generation; the traffic-pattern example; the fact that serving frameworks hide all of this; LRU and CPU answers.

5. Multi-head Latent Attention (MLA)

Summary. MLA (from DeepSeek) attacks the KV cache differently: instead of storing fewer K,V heads, it stores a **small compressed latent vector** per token and expands it back into K and V when attention is computed. It is about **attention optimization**, not KV management like PagedAttention [02:09:21]-[02:09:35].

5.1 The problem MLA still tackles

MHA, MQA and GQA all face the same issue: the KV cache consumes a lot of memory (especially for long-context LLMs), and each reduction in KV heads trades away some attention diversity [02:10:09]-[02:11:19]. MLA keeps the head structure but changes how K and V are represented.

5.2 The idea

KEY CONCEPT

GQA and MQA store **few KV heads**. MLA says: "why not change the entire representation?" Do not store the full K and V vectors; **compress them into a much smaller latent representation and store that** (PDF p. 18, [02:11:41]-[02:13:10]).

Latent simply means hidden: a compressed internal representation in a high-dimensional space that we cannot observe directly. Manisha's comparison with "hidden energy in physics" was accepted as fine [02:13:12]-[02:14:49].

Compression here = projection with a smaller matrix. Example: pass 8 numbers through a projection and get 3 numbers [02:27:34]-[02:28:25]. The matrix just changes the representation; the new size is smaller but still large enough (about 64-128, not 1 or 2) [02:48:13]-[02:48:49].

5.3 The numbers (per token)

The tutor recomputes yesterday's KV-memory numbers for one token with 8 heads of dimension 64 [02:18:46]-[02:23:25] (PDF p. 19-20):

Variant	K	V	Stored per token
MHA (8 KV heads)	8 x 64 = 512	512	1024
GQA (8 Q heads, 4 KV heads)	4 x 64 = 256	256	512
MQA (1 KV head)	64	64	128
MLA (assumed latent dimension 64)	one latent for both K and V		**64**

Derived from these class numbers: 64 / 1024 = 1/16 of MHA. This is a simple illustration, not a measured result [02:23:20]. These are per-token values; multiply by the number of tokens for a real cache size [02:21:34].

CONFLICT / AMBIGUITY

Latent dimension: 128 or 64? Paul says that in the original architecture the latent dimension is "generally 128, which is widely used" [02:18:07], and PDF p. 19 shows "Latent Dim = 128". The worked comparison then *assumes* 64 to stay consistent with the 64-dimension heads (PDF p. 20) [02:21:52]. *Used in these notes:* 128 as the typical real value, 64 for the numeric example.

5.4 How MLA works: compress, cache, expand

MLA pipeline (PDF p. 21-22)

```
input X
  |  compression (small matrix W_DKV)
  v
latent C_KV      <== this is what gets cached
  |  expansion (projection matrices)
  +--> K-related representation --+
  +--> V-related representation --+--> used in attention
per token you store: C1, C2, C3 ...  (instead of K1,V1  K2,V2 ...)
```

- **Compression:** $C_{KV} = X \times W_{DKV}$, where W_{DKV} is a "smaller matrix projection" (PDF p. 22, [02:32:14]-[02:33:17]).
- **Cache:** only the latent C_{KV} is cached, one latent per token (C1, C2, C3 ...) [02:30:58]-[02:31:06], [02:35:05]-[02:35:26].
- **Expansion (decompression):** when attention is computed, the latent is projected back up through another matrix, one for K and one for V, to recover the K and V needed by attention [02:33:33]-[02:34:28]. Paul calls it "crucial" because without it the dimensions would not work out [02:31:33]-[02:31:54]; the term "expanding" is used more often than "decompression" [02:39:04].
- **Mental model:** compress for storing, expand for computing. **Never store the full expanded KV cache** [02:39:34]-[02:40:00].

CONFLICT / AMBIGUITY

Matrix names. The board writes W_{DKV} for the compression matrix. For the expansion matrices PDF p. 22 shows $K = C_{KV}\ W_?$ and $V = C_{KV}\ W_?$ in handwriting where the subscripts are hard to read; PDF p. 19 also seems to write "$V = X\ W_Q$" (probably meant W_V). Paul's spoken version is garbled ("WDK... WDVK... another matrix, one for K, one for V") [02:32:40]-[02:33:57]. *Used in these notes:* one down-projection W_{DKV}, then one expansion matrix for K and one for V, names [unclear: exact subscripts]. Paul will redo the example in the next class [02:42:29], [02:49:05].

EXTRA CONTEXT (NOT FROM CLASS)

In the DeepSeek-V2 paper the matrices are named W^{DKV} (down-projection) and W^{UK}, W^{UV} (up-projections for keys and values). This is outside knowledge offered only to explain the board notation.

5.5 What changes and what does not

- **Queries are not compressed**: Q1 ... Q8 stay exactly the same; only K and V representation changes [02:23:35]-[02:24:44].
- **Heads still exist** (H1 ... H8). There are no "more KV heads"; the representation changed instead [02:24:02], [02:29:48]-[02:30:01]. All heads and all Q heads share the same latent [02:31:54]-[02:32:06], [02:38:41]-[02:39:04].
- **Compress the input, not the finished per-head KV.** The compression is applied directly on X when K,Q,V would be generated. Saying "we take all the KV heads and then convert them" is wrong [02:44:49]-[02:45:56], [02:50:06], [02:51:06].
- **One compression and one expansion**, not multiple layers of compression [02:35:59]-[02:36:22].
- MLA is **not about sharing** (GQA and MQA were); it is a different approach to the KV cache problem [02:26:01]-[02:26:21].

5.6 Quality and history

- According to the DeepSeek V2/V3 paper benchmarks the tutor sees **no loss, even an improvement**; this is specific to the datasets they used, and Paul says "follow the research" [02:28:57]-[02:29:40], [02:38:20]-[02:38:37].

- Paul calls MLA the **highest level of KV compression** among the techniques discussed so far [02:25:13]-[02:26:01].
- Started in **DeepSeek-V2**, discussed in the V3 paper, then used in R1 [02:26:21]-[02:26:53]. Next steps for DeepSeek: DSA (sparse attention) [02:56:01]-[02:56:08].

CONFLICT / AMBIGUITY

At the start Paul said MLA came in "DeepSeek 2.5 initially, later adopted with DeepSeek 3, 3.2" [00:04:50]. Later he said it started with V2, was described in the V3 paper and implemented in R1 [02:26:21]. *Used in these notes:* the later, more detailed statement.

5.7 RoPE preview (not taught yet)

- MLA is connected to RoPE: "about 50-60% of MLA is not related to RoPE but there is a 30% dependency" [01:32:33]. The DeepSeek paper brings RoPE into MLA [00:42:22].
- RoPE is **not** added at the embedding stage (token embedding + positional encoding). It is applied **inside self-attention** to inject the **relative position** of words [02:40:31]-[02:40:58]. PDF p. 22 shows "X P.E" and "TE + PE" crossed out, with an arrow to ROPE.
- RoPE is applied to queries and keys [02:42:46]-[02:42:57]. It is heavy maths (sine, cosine, rotation angles), so remember sin 0, cos 0, sin 90, cos 90 [01:34:53]-[01:35:19]. Expect to re-read it 4 to 5 times [01:31:56].

5.8 KV-cache quantization

A participant (Ramendra) asked if quantization can be used on the KV cache. Yes: the KV values are just numbers and can be converted, for example from 16-bit floating point to an integer type (transcribed as "intake"; [unclear: probably int8 or similar]) [02:47:03]-[02:47:22].

Q&A

Q. Is there a compression technique for K and V? [02:35:42]
A. Yes: bring in a smaller projection matrix and multiply; that is the only step.

Q. Can any dimension be compressed to any other? (Gaurav) [02:26:59]
A. Yes, like changing embedding dimension; compression to a higher or lower dimension is possible.

Q. Must the Q dimension align with the latent dimension? [02:28:38]
A. No. The 64 dimension stays the same in the inner dimension, so no extra alignment is needed.

Q. Is there information loss? [02:28:57]
A. The paper benchmarks show no loss and even improvement; dataset-specific.

Q. How many KV heads are there in MLA? [02:29:48]
A. No more heads; only the representation changed.

Q. Is there a decompression step? [02:30:01]
A. Yes; expansion back to K-related and V-related representations is needed for attention.

Q. Do all heads have separate latents? (Vivek) [02:31:54]
A. No, all heads use the same latent vector.

Q. Are there extra layers of compression? Will it hurt performance? (Ankit) [02:35:59]
A. No, only one compression and one expansion.

Q. Do we compress the KV of all eight heads? (Murtaza) [02:36:34]
A. Yes, into one shared latent; heads and Q are unchanged.

Q. Doesn't decompressing at runtime cancel the memory savings? (Gaurav) [02:36:57]

A. Paul says that is the wrong intuition. The saving is real because only the latent is stored and the full expanded KV is never stored. (His reply was short; no detail on compute cost was given.)

Q. Is PagedAttention possible with MLA? (Bhaskar) [02:37:09]

A. Yes; it works with MHA, MQA, GQA and MLA.

Q. Isn't C_{KV} already compressed? (Prahlad) [02:38:00]

A. Yes, it is the compressed/latent vector, later multiplied by a matrix.

Q. Will performance take a hit? (Yogi) [02:38:20]

A. Research shows no problem; performance was better.

Q. Does the latent KV exist at each head? (Mangesh) [02:38:41]

A. Yes: all heads and Q heads share that latent.

Q. Where do we decompress? [02:39:04]

A. When you need the original K and V representations for attention.

Q. Statement: "In MLA we take all the KV heads and convert into a single KV latent." (Murtaza) [02:44:49]

A. Correct that there is a single latent for all heads, but wrong that we take all KV heads first; compression is applied directly to X.

Q. For a token with 4 or 8 heads, do we take all KV heads and compress them? (Ramendra) [02:46:14], [02:51:06]

A. No. Same answer: smaller projection applied to X.

Q. Can KV be quantized? (Ramendra) [02:47:03]

A. Yes.

Q. I could not follow compression; what should I read? (Debajyoti) [02:47:34]

A. Read the DeepSeek paper (V3). Compression just means multiplying the input by a small matrix to get a lower-dimensional representation; Paul prefers the word "projection". He will calculate an example next class.

Q. Summary of the flow: prompt, prefill, KV cache, decode, new token, new KV... MQA/GQA/MLA diagrams (Alok) [02:50:09]

A. Paul confirmed all of it, including: MQA shares everything, GQA groups the query heads, and in MLA all query heads share one compressed latent KV.

Q. Is SVD a similar idea? (Anirudh) [02:50:37]

A. It is a good intuition because the idea is similar, but you cannot compare them directly.

Q. Is there a notebook? [02:42:17]

A. The MLA notebook is pushed to the repo but not complete; MLA will be continued next class together with RoPE.

Q. Is RoPE applied only to query and key? (Sai) [02:42:46]

A. Yes.

ADDITIONAL INFO FROM MATERIALS

PDF p. 21 shows the smaller-matrix idea ("8 nos to 3 nos") and the cache arrow at C_{KV}. PDF p. 22 contrasts old (token 1: K1,V1; token 2: K2,V2) with MLA (token 1: C1; token 2: C2; token 3: C3). PDF p. 23: Paged to MHA, MQA, GQA, MLA.

SAID IN CLASS, NOT IN MATERIALS

Latent means hidden; MLA is the best compression so far; the DeepSeek V2/V3/R1 history; KV quantization; the SVD comparison; the statement that RoPE is applied inside self-attention to Q and K.

6. Self-hosting and justifying GPU memory (MLOps question)

Summary. A short digression answering Mangesh's MLOps question [02:49:38]-[02:55:03]. It covers only a few sentences of new content.

- Do not mix MLOps with this topic: deep-learning model deployment differs from classical machine-learning deployment, which usually does not need a GPU [02:52:07]-[02:52:30].
- On managed cloud (for example Azure Machine Learning) you pick the compute and the platform manages the internals. When **self-hosting** you choose the GPU yourself, for example one with 180-200 GB VRAM at about $6 per hour, and must **justify** it by explaining how memory is split (model weights, KV cache, other processes) [02:53:22]-[02:54:17].
- Managed services cost more in the long run; self-hosting is harder at the start (especially bare metal) but cheaper long term, in Paul's view [02:54:31]-[02:54:58].

CONFLICT / AMBIGUITY

Example split here ("30% model, 50% KV cache") differs from the 65/30/5 example in section 3.7; see that box for the resolution.

Q&A

Q. Bedrock, SageMaker, Foundry: which to use? (Mangesh) [01:43:28]

A. Paul said the question was wrongly framed: each serves a different purpose; rephrase it better.

7. Key takeaways

1. All variants aim to shrink the KV cache: MHA (one K,V per head), MQA (one K,V total), GQA (one K,V per group), MLA (one small latent per token).
2. GQA keeps all query heads and the same dimension; it only reduces how many distinct K,V sets exist. KV memory remaining = KV heads / MHA heads.
3. Design rule: $H_{KV} = H_Q / 4$, sharing ratio 4 ("golden rule"); always a whole number; not a universal optimum.
4. $H_{KV} = H_Q$ is MHA and $H_{KV} = 1$ is MQA; GQA sits between.
5. Choose KV heads by looking at the KV-cache share of GPU memory (A100 40 GB example: 65% weights, 30% KV, 5% others); avoid OOM.
6. PagedAttention (vLLM) splits the KV cache into small fixed blocks mapped through a block table, so contiguous memory is no longer required.
7. Internal and external fragmentation are reduced, not eliminated; waste is at most about one block per sequence.
8. Identical prompt prefixes (for example a system prompt) share physical blocks; blocks are found by hash (prefix caching) and freed only when the reference count reaches 0.
9. Sharing KV does not make generated outputs identical; decode is non-deterministic.
10. MLA compresses the input with a small projection into a latent C_{KV}, caches only that, and expands it into K and V for attention; Q and the heads are unchanged, and the full expanded KV is never stored.

8. Glossary

Term	Meaning
KV cache	Stored keys and values of past tokens, reused so they are not recomputed.
MHA / MQA / GQA	Multi-Head / Multi-Query / Grouped-Query Attention: many, one, or grouped KV heads.
MLA	Multi-head Latent Attention (DeepSeek): caches a small latent instead of full K,V.
H_Q, H_{KV}	Number of query heads and of KV heads.
Sharing ratio	Q heads / KV heads (4 is the golden rule).
Attention diversity	How many different patterns the heads can capture.
Hyperparameter	A design choice fixed before training (KV heads, layers, hidden size, head dimension).
VRAM	GPU memory.
OOM	Out of memory error.
PagedAttention / vLLM	KV-cache management by blocks, from the vLLM serving project. SGLang is another serving framework.
Page / block / chunk	A small fixed-size unit of KV cache (16 tokens in the example).
Block table	Mapping from logical blocks to physical GPU blocks.
Contiguous memory	One unbroken region of memory.
Internal / external fragmentation	Allocated but unused space inside a block / free memory split into unusable scattered holes.
Prefill / decode	Computing the KV for the prompt / generating new tokens one by one.
Prefix caching, block sharing	Reusing the KV blocks of an identical prompt prefix, found by block hash.
Reference count	Number of sequences using a block; free only at 0.
System prompt (SP)	Fixed instruction text added in front of every user prompt.
Latent	Hidden, compressed internal representation.
C_{KV} / W_{DKV}	The cached latent vector / the small down-projection matrix producing it.
Projection	Multiplying by a matrix to change the representation or dimension.
RoPE	Rotary position encoding, applied inside attention to Q and K (not covered yet).

9. Action items, homework and logistics

- **Review** previous sessions if you missed them; yesterday's content is in Notion, code is in the same GitHub repo [00:01:15], [00:04:23].
- **Read about RoPE** before the next class: "take your time, go through RoPE, whatever content you can get" [02:41:44]. Recall sine/cosine values at 0 and 90 degrees [01:34:53].
- **Keep your own notes** for revision [01:36:00].
- **MLA notebook** is pushed but incomplete; MLA continues next class with the RoPE link and a worked W_{DKV} example, plus a look at the paper [02:42:17], [02:49:05], [02:55:11].
- **Questions** to Paul: Discord or the forums [02:56:29].
- **Next class (next Saturday, [02:56:29]):** MLA with RoPE, then Flash Attention (HBM, versions 1 and 2), PyTorch SDPA and scaling laws, ending Module 3; then Module 4 [01:35:29]-[01:36:52], [02:41:11]-[02:41:44].

- **Schedule:** Paul takes no classes from October 19 to November 10; a colleague (heard as "Divesh", and once "Ravi") will teach, starting with RAG versus fine-tuning, and Paul plans to start PEFT (LoRA/QLoRA) around October 17-18 [01:39:18]-[01:40:37]. Names are [unclear].
- **Later modules:** MoE, knowledge distillation, inference engineering (for example speculative decoding), deployment; other LoRA variants such as AdaLoRA [01:40:58]-[01:41:44]. More attention variants (gated attention, gated delta attention, KDA, DSA) belong to a future cohort [01:42:02], [02:55:44].

10. Resources and tools mentioned

- **vLLM** and **SGLang** (serving frameworks); **nvtop** / GPU monitoring tools.
- **DeepSeek V2 / V3 papers** (MLA source; Paul points to V3 [02:48:56]); the **GQA paper** (2023).
- Sebastian Raschka's content on attention variants (heard as "Rashka") [02:55:44].
- Notion (class content) and the GitHub repo (notebooks) [00:04:23].
- Hardware named: A100 (40 GB, 80 GB), H100 (80 GB), H200 (140 GB, as said); Azure Machine Learning, Bedrock, SageMaker, Foundry (in the MLOps question).
- Mentioned only in passing: Radix (not discussed [01:37:59]), Kimi attention (the heard phrase "KV Delta attention" is [unclear: probably Kimi Delta Attention]), DeepSeek sparse attention (DSA).

11. Unresolved or unclear points

- **Unknown sequence length:** Paul names it as problem 2 but never explicitly says how block allocation solves it.
- **Is Q ever cached / stored on the GPU?** Asked at [01:07:23]; Paul handed the question back without a stated answer.
- **LRU for pages:** "yes, but a different alternate is there" [01:06:31]; the alternative was not named.
- **MLA expansion cost:** the question whether runtime decompression hurts efficiency was dismissed as a wrong intuition without detail [02:36:57].
- **Exact MLA matrix names** and the worked example (postponed to next class).
- **Prime-number question:** the answer is partly garbled [00:34:23].
- **Yesterday's memory figure** ("for 10,000 tokens it was taking 6...") has no unit [02:02:10] [unclear].
- **Chat questions** are known only through Paul's spoken paraphrases; Ahmad and Aman did not respond when called [02:49:51], [02:55:04].
- **Names** of participants and of the covering instructor are spelled as heard.

12. Conflicts and ambiguities log

Each row shows what was found and which version these notes used.

#	Item	Versions found	Used in notes
1	GPU memory split	65% weights / 30% KV / 5% other (board, [00:37:35]) vs 30% model / 50% KV ([02:53:59])	65/30/5 (board example)
2	Usage of B (500 of 4096 tokens)	Said about 20% ([00:48:04])	About 12% (computed)
3	Which DeepSeek version introduced MLA	"2.5 then 3, 3.2" ([00:04:50]) vs V2, then V3 paper, then R1 ([02:26:21])	V2 / V3 / R1 (later statement)
4	Meaning of 12.5% (MQA vs MHA, 8 heads)	Reduction in one place, remaining share in another	Remaining share; reduction = 100% minus it
5	8 heads vs 32 heads in the memory example	Class starts with 8 and switches to 32 (PDF p. 3)	32-head table

#	Item	Versions found	Used in notes
6	Latent dimension	128 (typical; PDF p. 19) vs 64 (example)	128 typical; 64 for the numeric example
7	MLA matrix names	W_{DKV} clear; expansion names unreadable; "V = X W_Q" on PDF p. 19	W_{DKV} and generic K/V expansion matrices
8	Number of PagedAttention problems	Two on the board vs three in speech	Fragmentation (IF, EF), unknown length, duplication
9	Agenda order vs taught order	MLA second on agenda; PagedAttention taught first	Taught order
10	Model for the 65% example	"Llama 3, 13 billion" (unsure)	Mentioned as an illustration
11	Reference-count wording	Spoken: count "becomes true... 1... 0"; board 2, 1, 0	Board: 2, then 1, then 0
12	Token and key indices (K63/K64)	Off by one in places	16 consecutive keys per block
13	Paul wrote "3" for the number of heads in the recap	Corrected to 8 ([00:07:15])	8
14	MQA 1 of 32 heads	Said 3.12%	3.12% (1/32 = 3.125%)

13. Quick revision cheat sheet

Variant	KV heads (of 8 Q heads)	KV per token (64-dim heads, K+V)	Memory vs MHA
MHA	8	1024	100%
GQA (groups of 2)	4	512	50%
MQA	1	128	12.5%
MLA (latent 64, example)	latent	64	1/16 (derived)

KEY CONCEPT

GQA: $H_{KV} = H_Q / 4$ (sharing ratio 4); whole number; $H_{KV} = H_Q$ gives MHA, $H_{KV} = 1$ gives MQA.

32-head table: 32 KV = 100%, 16 = 50%, 8 = 25%, 4 = 12.5%, 1 = 3.12%.

GPU (A100 40 GB): 65% weights, 30% KV (~12 GB), ~5% other; keep 4-5% free; avoid OOM.

Memory aids

```
PagedAttention
 KV cache -> fixed blocks (16 tokens) -> block table -> any GPU block
 IF: waste only in last block (18 tokens = 16 + 2 -> 14 wasted)
 EF: scattered free blocks become usable (reduced, not solved)
 Sharing: same prefix -> same block (hash) ; refcount 2 -> 1 -> 0 -> free
 Clear shared cache when GPU ~95%; no TTL; outputs still non-deterministic

MLA
 X --W_DKV--> C_KV (cache only this) --expand--> K, V --> attention
 Q and heads unchanged; one compress + one expand; never store full KV
```

TIP

Interview hooks: define sharing ratio; explain internal vs external fragmentation with an example; explain why block sharing needs reference counts; explain what is cached in MLA and what is not.

Common mistakes: thinking Q is compressed in MLA; thinking heads or dimensions change in GQA; thinking PagedAttention is only for one attention type; thinking shared KV means shared outputs; "block 0 = GPU block 0".

Next class: MLA with RoPE and the W_{DKV} worked example, then Flash Attention, SDPA, scaling laws.